Date of the Session: _/_/_____ Time of The Session: ___to_____

#SKILL - 2 Hibernate CRUD Operations

Prerequisites:

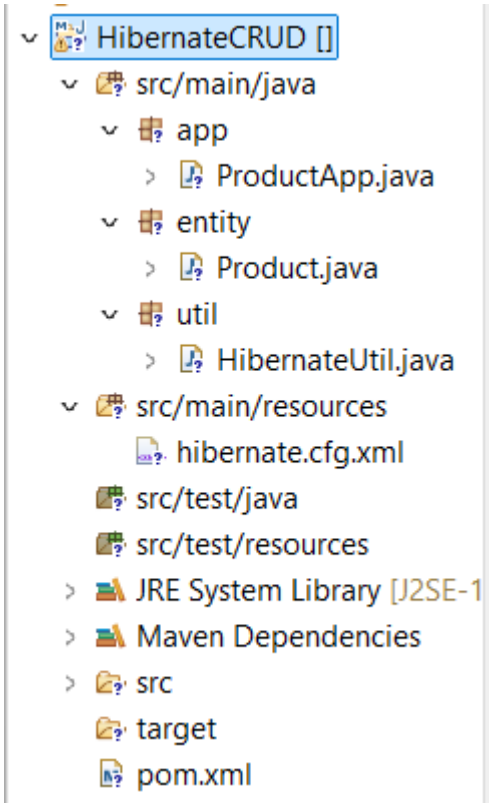
- Basic knowledge of Java
- Understanding of classes, objects, and OOP
- Familiarity with SQL basics

A retail inventory system needs to store product details such as productId, name, description, price, and quantity. The admin should be able to add new products, retrieve product information, update price or quantity, and delete discontinued products.

Your task is to implement complete CRUD operations using Hibernate with proper entity mapping using JPA annotations.

1. Create a Product entity with fields: id (primary key), name, description, price, quantity.
2. Configure Hibernate and map the Product entity to a table using JPA annotations.
3. Insert multiple Product records into the database.
4. Retrieve a product using its id.
5. Update the price or quantity of any selected product.
6. Delete a product record by id if it is discontinued.
7. Note:
    a. Implement Hibernate ID generation strategies (AUTO, IDENTITY, SEQUENCE) inside the entity to observe how primary key values differ.
    b. The ID can be manual or auto generated, based on your chosen strategy.
8. Push the Hibernate project into a single GitHub repository.

HibernateCRUD : Project Structure



ProductApp.java

```
package app;

import org.hibernate.Session;

import org.hibernate.Transaction;

import entity.Product;

import util.HibernateUtil;

public class ProductApp {

    public static void main(String[] args) {

        Session session = HibernateUtil.getSessionFactory().openSession();

        Transaction tx = session.beginTransaction();
```

```
        Product p1 = new Product("Laptop", "Dell i5", 55000, 10);
        Product p2 = new Product("Mouse", "Wireless", 600, 40);
        session.save(p1);
        session.save(p2);
        tx.commit();
        session.close();
        session = HibernateUtil.getSessionFactory().openSession();
        Product product = session.get(Product.class, 1);
        System.out.println(product.getName() + " " + product.getPrice());
        session.close();
        session = HibernateUtil.getSessionFactory().openSession();
        tx = session.beginTransaction();
        product = session.get(Product.class, 1);
        product.setPrice(58000);
        product.setQuantity(8);
        session.update(product);
        tx.commit();
        session.close();
        session = HibernateUtil.getSessionFactory().openSession();
        tx = session.beginTransaction();
        product = session.get(Product.class, 2);
        session.delete(product);
        tx.commit();
        session.close();

        System.out.println("CRUD Operations Completed");
    }
}
```

Product.java

```
package entity;
```

```java
import javax.persistence.*;
@Entity
@Table(name = "products")
public class Product {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private int id;
   private String name;
   private String description;
   private double price;
   private int quantity;
   public Product() {}
   public Product(String name, String description, double price, int quantity) {
      this.name = name;
      this.description = description;
      this.price = price;
      this.quantity = quantity;
   }
   public int getId() { return id; }
   public void setId(int id) { this.id = id; }
   public String getName() { return name; }
   public void setName(String name) { this.name = name; }
  public String getDescription() { return description; }
   public void setDescription(String description) { this.description = description;
  public double getPrice() { return price; }
   public void setPrice(double price) { this.price = price; }
   public int getQuantity() { return quantity; }
   public void setQuantity(int quantity) { this.quantity = quantity; }
}
```

HibernateUtil.java

```
package util;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
public class HibernateUtil {
private static final SessionFactory sessionFactory =
        new Configuration().configure().buildSessionFactory();
public static SessionFactory getSessionFactory() {
      return sessionFactory;
   }
}
```

Hibernate.cfg.xml

```
<!DOCTYPE hibernate-configuration>
<hibernate-configuration>
   <session-factory>
      <property name="hibernate.connection.driver_class">
         com.mysql.cj.jdbc.Driver
      </property>
      <property name="hibernate.connection.url">
         jdbc:mysql://localhost:3306/inventory_db
      </property>
      <property name="hibernate.connection.username">root</property>
      <property name="hibernate.connection.password">12345</property>
      <property name="hibernate.dialect">
         org.hibernate.dialect.MySQL8Dialect
      </property>
      <property name="hibernate.hbm2ddl.auto">update</property>
      <property name="hibernate.show_sql">true</property>
      <mapping class="entity.Product"/>
   </session-factory>
</hibernate-configuration>
```

Pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.hibernate</groupId>
  <artifactId>HibernateCRUD</artifactId>
  <version>0.0.1-SNAPSHOT</version>
     <dependencies>
       <dependency>
          <groupId>org.hibernate</groupId>
          <artifactId>hibernate-core</artifactId>
          <version>5.6.15.Final</version>
       </dependency>
       <dependency>
          <groupId>javax.persistence</groupId>
          <artifactId>javax.persistence-api</artifactId>
          <version>2.2</version>
       </dependency>
       <dependency>
          <groupId>mysql</groupId>
          <artifactId>mysql-connector-java</artifactId>
          <version>8.0.33</version>
       </dependency>
       </dependencies>
</project>
```

OUTPUTS:

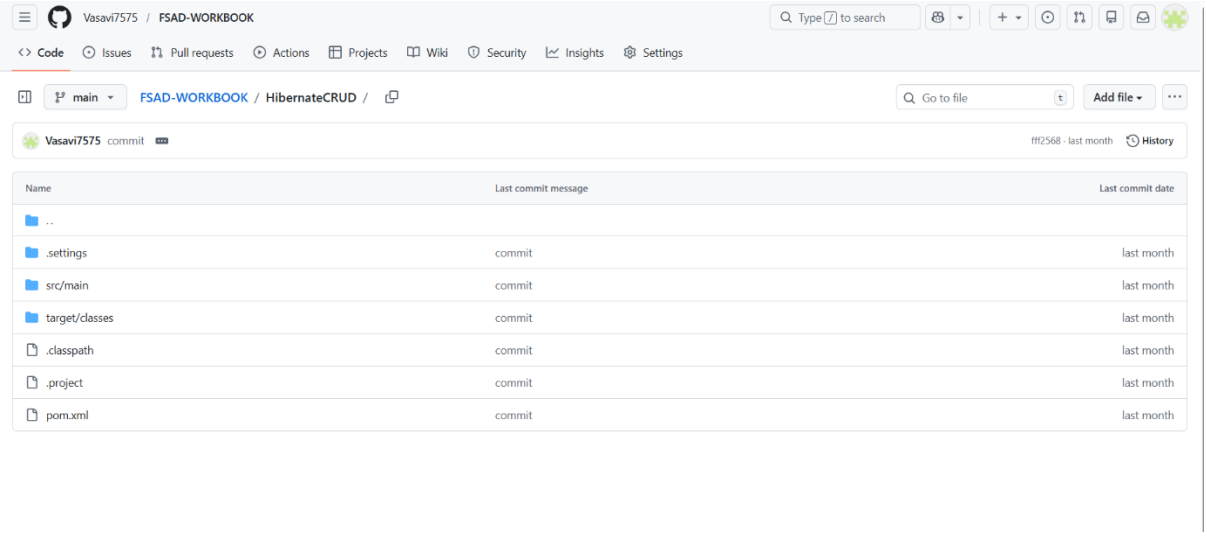
```
Console × Terminal
<terminated> ProductApp [Java Application] C:\Users\vasav\Downloads\sts-4.18.0.RELEASE\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.6.v20230204-1729\jre\bin\javaw.exe (18-Feb-2026, 10:
Hibernate: insert into products (description, name, price, quantity) values (?, ?, ?, ?)
Hibernate: insert into products (description, name, price, quantity) values (?, ?, ?, ?)
Hibernate: select product0_.id as id1_0_0_, product0_.description as descript2_0_0_, product0_.name as name3_0_0_, product0_.price as price4_
Laptop 58000.0
Hibernate: select product0_.id as id1_0_0_, product0_.description as descript2_0_0_, product0_.name as name3_0_0_, product0_.price as price4_
Hibernate: select product0_.id as id1_0_0_, product0_.description as descript2_0_0_, product0_.name as name3_0_0_, product0_.price as price4_
```

```
<terminated> ProductApp [Java Application] C:\Users\vasav\Downloads\sts-4.18.0.RELEASE\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.6.v20230204-17

product0_.price as price4_0_0_, product0_.quantity as quantity5_0_0_ from products product0_ where product0_.id=?

product0_.price as price4_0_0_, product0_.quantity as quantity5_0_0_ from products product0_ where product0_.id=?
product0_.price as price4_0_0_, product0_.quantity as quantity5_0_0_ from products product0_ where product0_.id=?
```

8.Push the Hibernate project into a single GitHub repository.



VIVA QUESTIONS:

1. **What is ORM and why is it used in application development?**
   ORM (Object-Relational Mapping) is a technique that maps Java objects to database tables.

Eliminates the need to write SQL manually for basic CRUD operations.
Converts data between **object-oriented** Java code and **relational databases**.
Examples: **Hibernate**, EclipseLink, OpenJPA.

2. Which annotation is used to mark a class as a JPA entity?

The annotation is:
@Entity
This tells JPA/Hibernate that the class should be mapped to a database table.

3. Why must every Hibernate entity have a primary key?

The primary key (@Id) uniquely identifies each entity in the database.
Hibernate uses the primary key to:
Track objects in memory (first-level cache).

4. How do you update an existing record using Hibernate?

Using `Session.update()`:

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();

Student student = session.get(Student.class, 1);
student.setName("New Name");
session.update(student);

tx.commit();
session.close();
```

(For Evaluator's use only)

Comment of the Evaluator (if Any)	**Evaluator's Observation** **Marks Secured:_____out of _** **EMP ID & Full Name of the Evaluator:** **Signature of the Evaluator Date of Evaluation:**

Date of the Session: _/_/_____ Time of The Session: __to_____